

1. Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using `fork()`. 3. Execute the command in the child process using an appropriate `exec()` system call. 4. Allow the parent process to wait for the child using `wait ()`. 5. Display the Process ID (PID) of both parent and child processes.

Using Linux terminal commands (`uname`, `lscpu`, `lsblk`, `ps`, `top`), investigate the relationship between hardware resources and operating system services.

Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

exp1.c

```
#include <stdio.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>

#include <stdlib.h>

int main()

{

    char command[100];

    printf("Enter a Linux command: ");

    scanf("%s", command);

    pid_t pid = fork();

    if (pid < 0)

    {

        perror("fork failed");
```

```
        return 1;
    }
    else if (pid == 0)
    {
        printf("Child Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        execlp(command, command, NULL);

        perror("exec failed");
        exit(1);
    }
    else
    {
        printf("Parent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        wait(NULL);

        printf("Child process completed.\n");
    }

    return 0;
}
```

Output :

```

charith@DESKTOP-RBQQQSS:~$ nano p41.c
charith@DESKTOP-RBQQQSS:~$ gcc p41.c -o p41
charith@DESKTOP-RBQQQSS:~$ ./p41
Enter a Linux command: ls
Parent Process
Parent PID: 750
Child PID: 751
Child Process
Child PID: 751
Parent PID: 750
OSSP      a.out      exp2      forkloop.c  multiplefork.c  pwfc5.c      skill3.c  zombie
Producer_consumer  childp8      exp2.c      hello      p41      server      skill4      zombie.c
Producer_consumer.c  childp8.c    exp3      hello.c    p41.c    server.c    skill4.c    zombie_fixed
Producer_consumer1  client      exp3.c      hello1      parentvschild  server_to_client  sql      zombie_fixed.c
Producer_consumer1.c  client.c    filename    hello1.c    parentvschild.c  shell      sq1
Task1      client_to_server  fork      input      ppc9      signal      sq2
Task1.c    copy.txt      fork.c      input.txt  pwc      signal.c    sq3
Task2      display      fork.c      input.txt  pwc      signal.c    sq3
Task2.c    display.c    forkloop    multiplefork  pwfc5      skill3      sq3.c
Child process completed.

```

Develop a C program that uses the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution.

Use the `strace` utility to trace all system calls generated by the command `cat sample.txt`. Analyze the sequence of system calls and identify the kernel

services involved. **exp2.c**

```

#include <stdio.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>

#include <stdlib.h>

int main()
{
    pid_t pid;

    printf("Before fork():\n");

    printf("Process PID : %d\n", getpid());

    printf("Parent PPID : %d\n", getppid());

    pid = fork();

    if (pid < 0)
    {

```

```

    perror("fork failed");
    return 1;
}

else if (pid == 0)
{
    // Child process printf("\n--- CHILD
    PROCESS ---\n"); printf("Child PID    :
    %d\n", getpid()); printf("Child PPID   :
    %d\n", getppid()); printf("Child State :
    Running\n");

    sleep(5);

    printf("Child State : Terminating\n");
    exit(0);
}

else
{
    // Parent process printf("\n--- PARENT
    PROCESS ---\n"); printf("Parent PID    :
    %d\n", getpid()); printf("Parent PPID :
    %d\n", getppid()); printf("Child PID :
    %d\n", pid); printf("Parent State :
    Running\n");

    printf("\nParent waiting for child...\n");
    wait(NULL);
}

```

```

printf("Child terminated.\n"); printf("Parent State :
Running\n"); printf("Parent State : Terminated after
completion\n");
}

return 0;
}

```

Output :

```

charith@DESKTOP-RBQQSS:~$ nano p42.c
charith@DESKTOP-RBQQSS:~$ gcc p42.c -o p42
charith@DESKTOP-RBQQSS:~$ ./p42
Before fork():
Process PID : 821
Parent PPID : 318

--- PARENT PROCESS ---
Parent PID : 821
Parent PPID : 318
Child PID : 822
Parent State : Running

Parent waiting for child...

--- CHILD PROCESS ---
Child PID : 822
Child PPID : 821
Child State : Running
Child State : Terminating
Child terminated.
Parent State : Running
Parent State : Terminated after completion

```

Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations. **exp3.c**

```
#include <stdio.h>
```

```
#include <unistd.h>

#include <sys/types.h>


int main()
{
    pid_t pid;


    printf("Before fork()\n");
    printf("Process PID: %d\n", getpid());


    pid = fork();


    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }


    if (pid == 0)
    {
        printf("\nChild Process\n");
        printf("PID : %d\n", getpid());
        printf("PPID : %d\n", getppid());


        sleep(10);
```

```

        printf("Child process terminating...\n");
    }
    else
    {
        printf("\nParent Process\n");
        printf("PID : %d\n", getpid());
        printf("PPID : %d\n", getppid());

        sleep(5);

        printf("Parent process terminating...\n");
    }

    return 0;
}

```

Output:

```

charith@DESKTOP-RBQQSS:~$ nano p43.c
charith@DESKTOP-RBQQSS:~$ gcc p43.c -o p43
charith@DESKTOP-RBQQSS:~$ ./p43
Before fork()
Process PID: 862

Parent Process
PID : 862
PPID : 318

Child Process
PID : 863
PPID : 862
Parent process terminating...

```

4. Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

exp4.c

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <sys/types.h>
```

```
#include <sys/wait.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{ int i; pid_t
```

```
    pid;
```

```
    for (i = 0; i < 3; i++)
```

```
    {
```

```
        pid = fork();
```

```
        if (pid < 0)
```

```
        {
```

```
            perror("fork failed");
```

```
            exit(1);
```

```
        }
```

```
        if (pid == 0)
```

```
        {
```

```

        printf("Child %d: PID = %d, PPID = %d\n",
               i + 1, getpid(), getppid());

        sleep(i + 1);

        printf("Child %d completed.\n", i + 1);
        exit(0);
    }
}

for (i = 0; i < 3; i++)
{
    wait(NULL); printf("Parent
    collected a child.\n");
}

printf("All children completed.\n");

return 0;
}

```

```

charith@DESKTOP-RBQQSS:~$ nano p44.c
charith@DESKTOP-RBQQSS:~$ gcc p44.c -o p44
charith@DESKTOP-RBQQSS:~$ ./p44
Child 1: PID = 926, PPID = 925
Child 2: PID = 927, PPID = 925
Child 3: PID = 928, PPID = 925
Child 1 completed.
Parent collected a child.
Child 2 completed.
Parent collected a child.
Child 3 completed.
Parent collected a child.
All children completed.

```